Versi 1.0

September 2026

ALGORITMA PEMROGRAMAN

MODUL 2 - Insertion Sort & Shell Sort

Author:

Nur Hayatin, S.ST., M.Kom. Ph.D

Muhammad Ibrahim Al Ayubi

Mareno Ali Ibrahim

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
PENDAHULUAN **3**
TUJUAN 3
TARGET MODUL 3
PERSIAPAN 3
KEYWORDS 3
CONTENTS **4**
1. Insertion Sort 4
1.1. Definisi Insertion Sort 4
1.2. Cara Kerja Insertion Sort 4
1.3. Analisis Kompleksitas Waktu Insertion Sort 7
1.4. Kelebihan Insertion Sort 7
1.5. Kekurangan Insertion Sort 8
2. Shell Sort 8
3.1. Definisi Shell Sort 8
3.2. Cara Kerja Shell Sort 8
3.3. Analisis Kompleksitas Waktu Shell Sort 12
3.4. Kelebihan Shell Sort 12
3.5. Kekurangan Shell Sort 12
CODELAB **14**
DEMO **15**
Assessment **17**
Assessment Rubric 17
Rating Scale 17

PENDAHULUAN

TUJUAN

1. Membantu mahasiswa memahami konsep dasar algoritma pengurutan dalam pemrograman.
2. Memperkenalkan cara pengorganisasian data menggunakan teknik iteratif sederhana.
3. Memberikan pengalaman praktis kepada mahasiswa dalam mengimplementasikan algoritma pengurutan menggunakan Java.

TARGET MODUL

1. Mahasiswa mampu memahami cara kerja Insertion Sort dan Shell Sort.
2. Mahasiswa mampu mengimplementasikan algoritma pengurutan menggunakan bahasa pemrograman Java.
3. Mahasiswa mampu menganalisis efisiensi algoritma pengurutan menggunakan kompleksitas waktu (Big-O).

PERSIAPAN

1. Mahasiswa muslim dapat membaca doa sebelum memulai pelajaran.

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop atau komputer pribadi.
3. IntelliJ IDE atau Java IDE lainnya (Eclipse/VSCode).
4. Motivasi kuat dan komitmen untuk belajar.

KEYWORDS

Sorting, Insertion Sort, Shell Sort, Iteration, Array, Time Complexity, Java Programming

CONTENTS

1. Insertion Sort

1.1. Definisi Insertion Sort

Insertion sort adalah algoritma pengurutan berbasis perbandingan yang membangun urutan terurut satu elemen dalam satu waktu. Array secara konseptual dibagi menjadi dua bagian: bagian terurut yang terus bertumbuh di setiap iterasi, dan bagian belum terurut yang terus menyusut secara bersamaan.

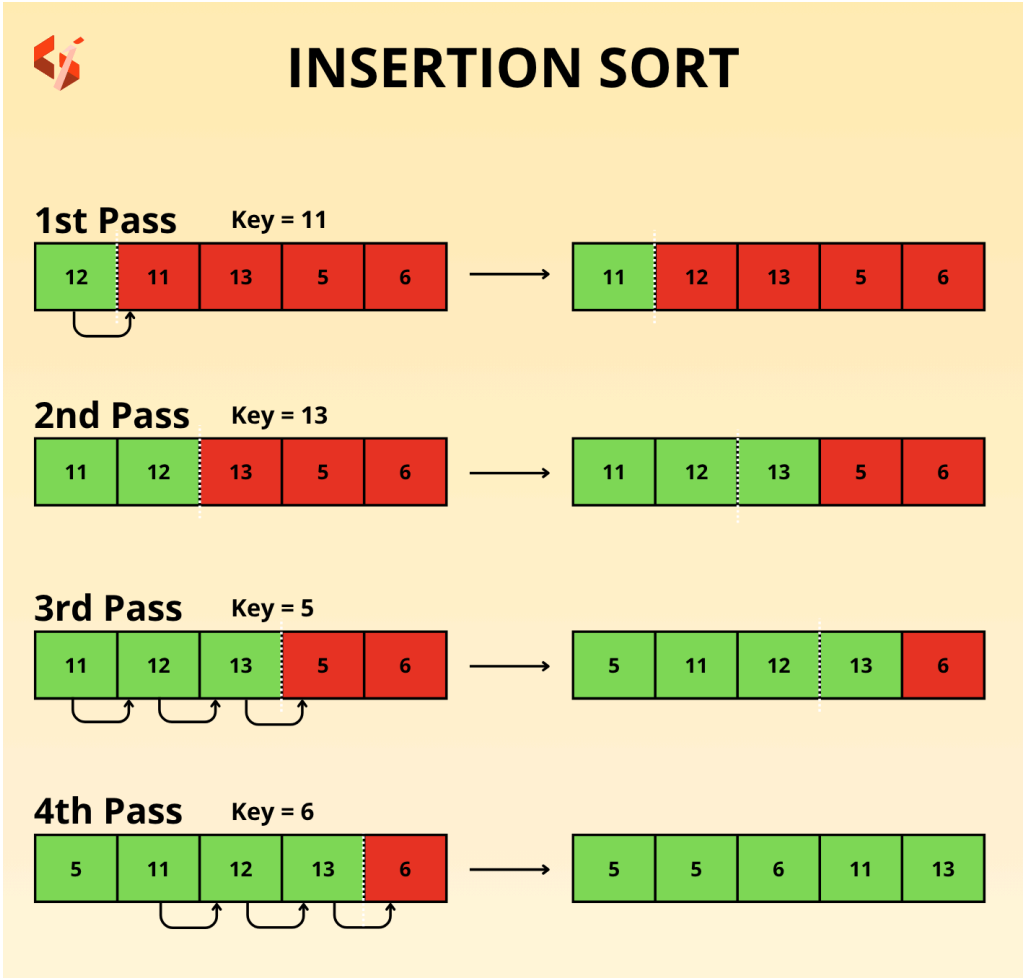
Pada awal proses, bagian terurut hanya terdiri dari satu elemen, yaitu elemen pertama array, karena urutan yang hanya terdiri dari satu elemen sudah pasti terurut. Algoritma kemudian memproses elemen-elemen berikutnya satu per satu. Untuk setiap elemen, ia dibandingkan dengan elemen-elemen pada bagian terurut, bergerak dari kanan ke kiri, hingga menemukan posisi yang tepat. Elemen-elemen yang lebih besar dari elemen yang sedang disisipkan digeser satu posisi ke kanan untuk memberi ruang, setelah itu elemen ditempatkan pada posisi yang sesuai.

Prosedur ini menyerupai cara seseorang menyortir kartu remi. Setiap kartu baru yang diambil dibandingkan dengan kartu-kartu yang sudah tersusun, lalu disisipkan ke posisi yang tepat, daripada mengatur ulang seluruh kartu dari awal.

Insertion sort bekerja sangat baik pada kumpulan data berukuran kecil dan data yang hampir terurut, karena lebih sedikit pergeseran yang diperlukan. Namun, efisiensinya menurun pada kumpulan data yang lebih besar dan tidak terurut, karena jumlah perbandingan dan pergeseran meningkat secara signifikan seiring bertambahnya ukuran masukan.

1.2. Cara Kerja Insertion Sort

Insertion Sort bekerja dengan membagi array menjadi dua bagian: bagian terurut dan bagian belum terurut. Pada awalnya, elemen pertama dianggap sudah terurut. Algoritma kemudian mengambil elemen berikutnya dari bagian belum terurut dan membandingkannya dengan elemen-elemen pada bagian terurut untuk menemukan posisi yang tepat. Elemen-elemen yang lebih besar digeser satu posisi ke kanan hingga algoritma menemukan posisi yang benar untuk elemen saat ini. Proses ini diulang hingga seluruh elemen terurut.



Gambar 1. Ilustrasi Insertion Sort

1. Mulai dari elemen kedua, karena elemen pertama dianggap sudah terurut.
2. Bandingkan dan tukar elemen kedua dengan elemen pertama jika elemen kedua lebih kecil.
3. Pindah ke elemen ketiga, bandingkan dengan dua elemen sebelumnya, dan tempatkan pada posisi yang tepat.
4. Ulangi hingga seluruh array terurut.

Code Implementation:

```java
public class InsertionSort {
    /* Function to sort array using insertion sort */
    void sort(int arr[])
    {
        int n = arr.length;
        for (int i = 1; i < n; ++i) {
            int key = arr[i];
            int j = i - 1;

            /* Move elements of arr[0..i-1], that are greater than key,
               to one position ahead of their current position */
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j = j - 1;
            }
            arr[j + 1] = key;
        }
    }

    /* A utility function to print array of size n */
    static void printArray(int arr[])
    {
        int n = arr.length;
        for (int i = 0; i < n; ++i)
            System.out.print(arr[i] + " ");

        System.out.println();
    }

    // Driver method
    public static void main(String args[])
    {
        int arr[] = { 12, 11, 13, 5, 6 };

        InsertionSort ob = new InsertionSort();
        ob.sort(arr);

        printArray(arr);
    }
}
```

1.3. Analisis Kompleksitas Waktu Insertion Sort

Kompleksitas Waktu

Kasus	Kompleksitas	Penjelasan
Kasus Terbaik	O(n)	Terjadi ketika array sudah terurut. Algoritma hanya memerlukan satu perbandingan per elemen dan tidak melakukan pergeseran sama sekali.
Kasus Rata-rata	$O(n^2)$	Elemen-elemen perlu dibandingkan dan digeser beberapa kali untuk mencapai posisi yang tepat.
Kasus Terburuk	$O(n^2)$	Terjadi ketika array terurut secara terbalik. Setiap elemen harus dibandingkan dengan semua elemen sebelumnya dan digeser berulang kali.

Kompleksitas Ruang

Jenis Kompleksitas	Nilai	Penjelasan
Kompleksitas Ruang	O(1)	Insertion Sort mengurutkan secara langsung (in-place) hanya menggunakan sejumlah kecil memori tambahan yang konstan, terlepas dari ukuran masukan.

1.4. Kelebihan Insertion Sort

- Mudah Diimplementasikan
  Salah satu algoritma pengurutan yang paling mudah dipahami dan dikodekan, sehingga ideal untuk pemula atau aplikasi berskala kecil.
- Efisien untuk Data Kecil atau Hampir Terurut
  Bekerja sangat baik pada kumpulan data kecil atau array yang sudah hampir terurut, mendekati O(n) dalam kondisi tersebut.
- Pengurutan Stabil
  Mempertahankan urutan relatif elemen-elemen yang bernilai sama, yang berguna saat mengurutkan data dengan banyak bidang.
- Pengurutan Dalam Tempat (In-place)
  Hanya membutuhkan O(1) memori tambahan karena mengurutkan array secara langsung tanpa memerlukan struktur data tambahan.

- Algoritma Adaptif
  Secara otomatis memanfaatkan urutan yang sudah ada dalam data, sehingga mengurangi jumlah perbandingan dan pergeseran yang diperlukan.

1.5. Kekurangan Insertion Sort

- Tidak Efisien untuk Data Berukuran Besar
  Dengan kompleksitas kasus terburuk $O(n^2)$, performa menurun secara signifikan seiring bertambahnya jumlah elemen.
- Jumlah Pergeseran yang Tinggi
  Setiap penyisipan mungkin memerlukan pergeseran banyak elemen satu posisi dalam satu waktu, sehingga lambat pada array berukuran besar atau terurut terbalik.
- Tidak Cocok untuk Aplikasi Berskala Besar
  Seiring bertambahnya data, kompleksitas $O(n^2)$ membuat Insertion Sort tidak praktis untuk aplikasi dunia nyata yang menangani data bervolume besar.
- Performa Buruk pada Data Terurut Terbalik
  Ketika array terurut secara terbalik, setiap elemen harus dibandingkan dan digeser hingga ke posisi paling depan, mengakibatkan biaya waktu yang maksimal.
- Lebih Lambat Dibandingkan Algoritma Modern
  Dibandingkan dengan algoritma seperti Merge Sort atau Quick Sort, Insertion Sort jauh lebih lambat dan kurang skalabel untuk penggunaan umum.

2. Shell Sort

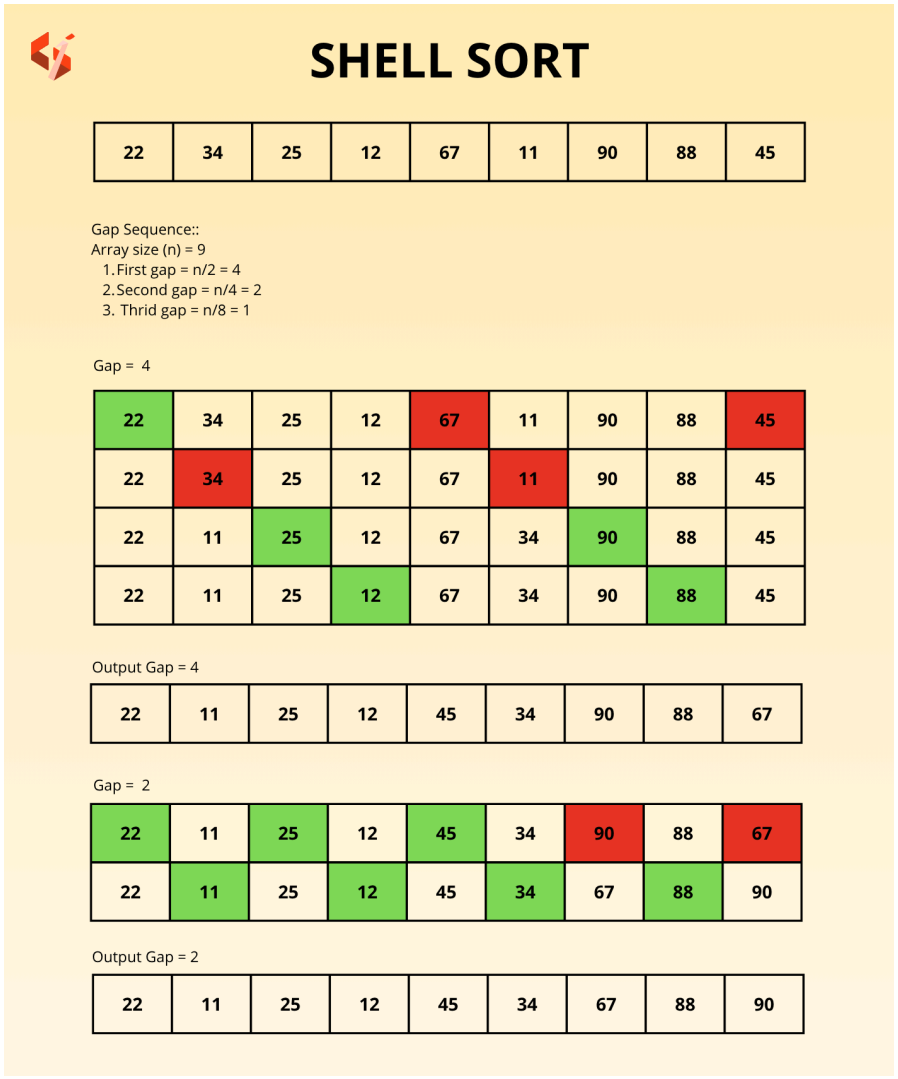
3.1. Definisi Shell Sort

Shell Sort, yang juga dikenal sebagai Metode Shell, adalah versi yang disempurnakan dari Insertion Sort yang membandingkan dan mengurutkan elemen-elemen yang berjauhan menggunakan nilai jarak (gap) tertentu. Algoritma secara bertahap mengurangi nilai gap hingga mencapai 1, di mana tahap akhir bekerja seperti Insertion Sort biasa. Dengan memungkinkan elemen berpindah jarak jauh lebih awal dalam proses pengurutan, Shell Sort dapat meningkatkan efisiensi secara signifikan dibandingkan Insertion Sort standar, terutama untuk kumpulan data yang lebih besar. Shell Sort adalah algoritma pertama yang berhasil melampaui batas kompleksitas waktu $O(n^2)$ dalam pengurutan.

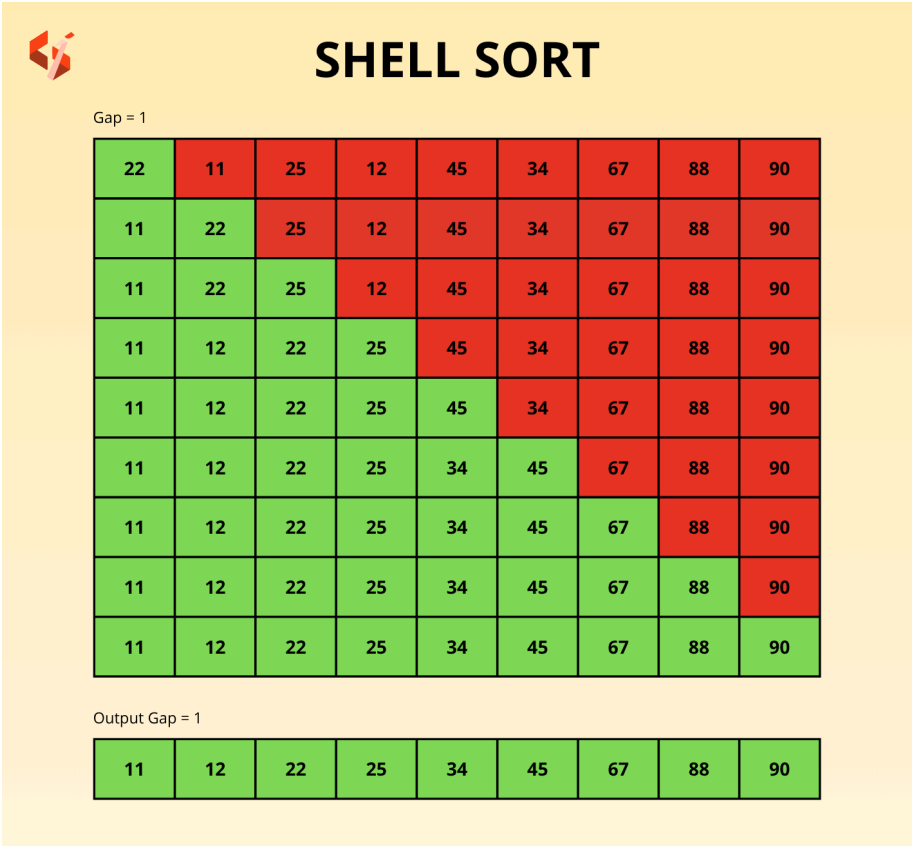
3.2. Cara Kerja Shell Sort

Shell Sort membagi proses pengurutan menjadi beberapa tahap menggunakan nilai gap. Pada awalnya, gap diatur ke nilai besar, biasanya setengah dari ukuran array. Elemen-elemen yang dipisahkan oleh jarak gap dibandingkan dan diurutkan menggunakan metode yang serupa dengan Insertion Sort. Setelah setiap tahap, gap berkurang secara bertahap hingga mencapai 1.

Ketika gap mencapai 1, algoritma melakukan Insertion Sort terakhir pada array yang sudah hampir terurut. Karena banyak elemen sudah berpindah mendekati posisi yang benar selama tahap-tahap sebelumnya, pengurutan akhir menjadi jauh lebih cepat dan efisien.



Gambar 2.1 Ilustrasi Shell Sort



Gambar 2.2 Ilustrasi Shell Sort

1. Pilih urutan gap (umumnya n/2, n/4, ..., 1).
2. Urutkan elemen pada setiap gap menggunakan Insertion Sort.
3. Kurangi gap dan ulangi hingga gap mencapai 1.

Code Implementation:

```java
public class ShellSort {

    public static void shellSort(int[] arr) {
        int n = arr.length;
        // Start with a large gap, then reduce it step by step
        for (int gap = n / 2; gap > 0; gap /= 2) {

            // Perform a "gapped" insertion sort for this gap size
            for (int i = gap; i < n; i++) {

                // Current element to be placed correctly
                int temp = arr[i];
                int j = i;

                // Shift earlier elements that are greater than temp
                while (j >= gap && arr[j - gap] > temp) {
                    arr[j] = arr[j - gap];
                    j -= gap;
                }

                // Place temp in its correct position
                arr[j] = temp;
            }
        }
    }
    // Utility function to print array
    public static void printArray(int[] arr) {
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr = {12, 34, 54, 2, 3};

        shellSort(arr);
        printArray(arr);
    }
}
```

3.3. Analisis Kompleksitas Waktu Shell Sort

Kompleksitas Waktu

Kasus	Kompleksitas	Penjelasan
Kasus Terbaik	O(n log n)	Array sudah terurut sehingga elemen hanya memerlukan perbandingan minimal dan tidak ada pergeseran di seluruh urutan gap.
Kasus Rata-rata	Bergantung pada gap	Elemen berada dalam urutan acak, memerlukan beberapa tahap dengan gap yang semakin berkurang, sehingga mengurangi jumlah pergeseran yang diperlukan.
Kasus Terburuk	$O(n^2)$	Array terurut secara terbalik, menyebabkan perbandingan dan pergeseran maksimal di setiap urutan gap hingga tahap akhir.

Kompleksitas Ruang

Jenis Kompleksitas	Nilai	Penjelasan
Kompleksitas Ruang	O(1)	Shell Sort mengurutkan secara langsung (in-place) hanya menggunakan sejumlah kecil memori tambahan yang konstan, terlepas dari ukuran masukan.

3.4. Kelebihan Shell Sort

- Lebih Cepat dari Insertion Sort pada Data Besar
  Dengan mengurutkan elemen yang berjauhan terlebih dahulu, Shell Sort secara signifikan mengurangi total jumlah pergeseran dibandingkan Insertion Sort.
- Pengurutan Dalam Tempat (In-place)
  Hanya membutuhkan O(1) memori tambahan karena mengurutkan array secara langsung tanpa memerlukan struktur data tambahan.
- Cocok untuk Kumpulan Data Berukuran Sedang
  Bekerja efisien pada array berukuran sedang di mana algoritma sederhana terlalu lambat tetapi algoritma kompleks belum diperlukan.
- Perilaku Adaptif
  Seperti Insertion Sort, Shell Sort memanfaatkan data yang sudah sebagian terurut dan mengurangi operasi yang tidak perlu ketika urutan sudah ada.

3.5. Kekurangan Shell Sort

- Pemilihan Urutan Gap yang Kompleks
  Performa sangat bergantung pada urutan gap yang dipilih, dan tidak ada urutan yang secara universal optimal untuk semua kasus.

- Bukan Pengurutan Stabil
  Shell Sort tidak mempertahankan urutan relatif elemen-elemen yang bernilai sama, yang dapat menjadi masalah saat mengurutkan data dengan banyak bidang.
- Kompleksitas Waktu yang Tidak Konsisten
  Kompleksitas rata-rata dan kasus terburuk bervariasi bergantung pada urutan gap yang digunakan, sehingga performa lebih sulit diprediksi.
- Sulit Dianalisis Secara Teoretis
  Kompleksitas waktu pasti dari Shell Sort masih belum sepenuhnya dipahami untuk semua urutan gap, sehingga lebih sulit dianalisis dibandingkan algoritma lain.
- Kurang Kompetitif Dibandingkan Algoritma Modern
  Untuk kumpulan data berukuran besar, Shell Sort umumnya dikalahkan oleh algoritma yang lebih canggih seperti Merge Sort atau Quick Sort

CODELAB

Insertion Sort & Shell Sort

Tujuan: Implementasikan Bubble Sort dan Selection Sort dalam urutan menurun dengan melengkapi bagian yang kosong pada program.

Tugas Anda

Bubble Sort:

1. Lengkapi kondisi batas perulangan
2. Lengkapi kondisi pergeseran untuk urutan menurun
3. Lengkapi langkah pergerakan indeks

Selection Sort:

1. Lengkapi nilai gap awal
2. Lengkapi aturan pengurangan gap
3. Lengkapi kondisi perbandingan untuk urutan menurun
4. Lengkapi langkah pergeseran berdasarkan jarak gap

Instructions:

1. Clone repositori dari GitHub di terminal Anda:
   "git clone -b modul2 https://github.com/KumaReno/Praktikum-Alpro"
2. Buka proyek menggunakan Java IDE Anda
3. Lengkapi semua bagian yang kosong (____)
4. Jalankan program dan verifikasi keluaran

DEMO

Z-App: Building a Recommendation & Digital Trend Engine

Project Overview

Proyek demo ini melanjutkan pengembangan platform Z-App dari Modul 1. Sebelumnya, Z-App mengimplementasikan Bubble Sort dan Selection Sort untuk mengorganisasi data postingan terbaru berdasarkan popularitas. Namun, seiring platform berkembang dengan pesat, sistem kini perlu memproses kumpulan data yang lebih besar, seperti peringkat influencer dengan ribuan data pengikut.

Karena algoritma sebelumnya menjadi kurang efisien untuk kumpulan data yang lebih besar, tim pengembang memutuskan untuk meningkatkan mesin peringkat dengan menambahkan:

- Insertion Sort
- Shell Sort

Dalam modul ini, mahasiswa akan melanjutkan pengembangan sistem Z-App dengan mengimplementasikan algoritma pengurutan iteratif yang lebih canggih serta membandingkan performanya pada berbagai ukuran kumpulan data.

Scenario & Requirements

Anda masih merupakan bagian dari tim pengembang Z-App.

Setelah berhasil mengurutkan popularitas postingan di Modul 1, platform kini memperkenalkan fitur baru bernama "Top 100 Influencer". Fitur ini mengurutkan influencer berdasarkan total pengikut. Karena kumpulan data jauh lebih besar dari yang digunakan di Modul 1, tim pengembang memerlukan algoritma yang lebih efisien.

Mahasiswa diwajibkan untuk mengimplementasikan:

- Insertion Sort
- Shell Sort

Sistem harus mendukung:

- Ascending Order
- Descending Order

Proses pengurutan juga harus menampilkan setiap iterasi/tahap untuk memvisualisasikan cara kerja algoritma secara internal.

Dataset Requirements

- Dataset yang digunakan dalam modul ini harus berbeda dari dataset yang digunakan di Modul 1
- Dataset harus merepresentasikan jumlah pengikut influencer

- Ukuran dataset harus 100 atau lebih

Tasks

1. Buat kelas InsertionSort.java. Kelas ini harus memuat:
    - Metode Insertion Sort urutan menaik
    - Metode Insertion Sort urutan menurun
    - Visualisasi setiap iterasi/tahap selama proses pengurutan
2. Buat kelas ShellSort.java. Kelas ini harus memuat:
    - Metode Shell Sort urutan menaik
    - Metode Shell Sort urutan menurun
    - Visualisasi setiap iterasi/tahap selama proses pengurutan
3. Perbarui kelas Main.java sebelumnya:
    - Pertahankan fitur pengurutan dari Modul 1
    - Tampilkan sistem menu
    - Tambahkan dukungan untuk Insertion Sort dan Shell Sort
    - Izinkan pengguna memilih algoritma pengurutan
    - Tampilkan:
        - Array asli
        - Keluaran iterasi/tahap
        - Hasil akhir yang terurut
    - Izinkan program berjalan berulang tanpa perlu dimulai ulang (Opsional)

Notes:

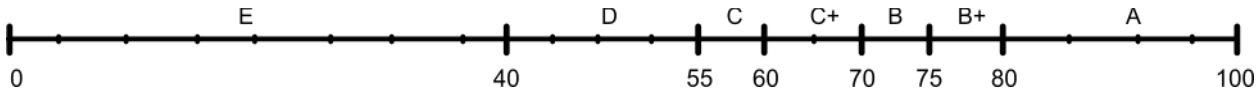
- Jangan gunakan metode pengurutan bawaan seperti Arrays.sort()
- Program harus diimplementasikan secara manual
- Pastikan setiap iterasi/tahap ditampilkan dengan jelas

Assessment

Assessment Rubric

Aspek Penilaian	Point
Codelab	**Total 20%**
DEMO	**Total 80%**
Output Accuracy	10%
Implementation (Insertion Sort)	20%
Implementation (Shell Sort)	20%
Answer Questions	30%

Rating Scale



SSS = (81 - 100) → SSSensational!!!

SS = (75 - 80) → Sick Skills!!

S = (70 - 74) → Sweet!

A = (60 - 69) → Alright

B = (55 - 59) → Bravo!

C = (41 - 54) → Come on!

D = (0 - 40) → Dull